

Incubyte Data Engineer — Technical Assessment

Expectations

This exercise is intentionally designed to assess how you think, design, and build software in an AI-driven environment.

You are required to use AI tools to accelerate your work. We care about how you use them, the clarity of your thinking, and the quality of your engineering decisions.

We expect you to:

- Demonstrate clarity in thought and structured problem solving
- Show strong engineering fundamentals & product thinking
- Make thoughtful architectural and design decisions
- Write production-quality code and tests
- Use AI intentionally while maintaining correctness and quality

Please make incremental commits so we can understand how your solution evolved.

Problem Statement:

I run a global airline loyalty program called SkyPoints, with lounges and partner airlines across the world. Every member enrolled in the program is issued a Membership Card that lets them access any partner lounge worldwide, redeem miles, and track their tier status.

Current Status:

We maintain all members in one database. There are millions of members enrolled in the program. So, I decided to split up the members based on the country and load them into corresponding country tables.

To pull the members as per Country, my developers should know what are all the places the Member Data is available. So, the data extraction will be done by our Source System. It will pull all the relevant member data and give us two feeds every day: a flat file of member profile data, and a semi-structured JSON feed of mileage redemption transactions from our partner airlines.

In design documents, you will have:

- File Name Specification – Name String, Extension of the files
- Date and Time format of the File – YYYYMMDD, HHMMSS TT or any other format
- Header Record Layout –
|H|Member_Name|Member_Id|Enrollment_Date|Last_Flight_Date|Tier_Code|Agent_Name|State|Country|DOB|Is_Active
- Detail Record Layout – |D|Elena|223457|20101012|20121013|GLD|Sam|CA|USA|03051985|A

Detail Records will tell you what data you are getting from source, what data type, is it mandatory or not, and the length of the column.

File Position	Column Name	Field Length	Data Type	Mandatory	Key Column
1	Member Name	255	VARCHAR	Y	Y
2	Member ID	18	VARCHAR	Y	N
3	Enrollment Date	8	DATE	Y	N
4	Last Flight Date	8	DATE	N	N
5	Tier Code	5	CHAR	N	N
6	Agent Name	255	CHAR	N	N
7	State	5	CHAR	N	N
8	Country	5	CHAR	N	N
9	Post Code	5	INT	N	N
10	Date of Birth	8	DATE	N	N
11	Active Member	1	CHAR	N	N

The sample flat file will be:

```
H|Member_Name|Member_Id|Enrollment_Date|Last_Flight_Date|Tier_Code|Agent_Name|State|Country
|DOB|Is_Active
|D|Elena|223457|20101012|20121013|GLD|Sam|CA|USA|03051985|A
|D|Ravi|223458|20101012|20121013|SLV|Sam|TN|IND|03051985|A
|D|Mateo|223459|20101012|20121013|GLD|Sam|NCR|PHIL|03051985|A
|D|Nora|22345|20101012|20121013|PLT|Sam|ONT|CAN|03051985|A
|D|Jacob|2256|20101012|20121013|SLV|Sam|VIC|AU|03051985|A
```

Alongside the flat file, the partner airline sends a daily JSON feed of redemption transactions:

```
{
  "member_id": "223457",
  "feed_date": "20240115",
  "redemptions": [
    { "txn_id": "RX10091", "txn_date": "20240110", "partner": "AeroLink", "miles_redeemed": 12000, "status": "COMPLETED" },
    { "txn_id": "RX10092", "txn_date": "20240113", "partner": "SkyPoints", "miles_redeemed": 5000, "status": "PENDING" }
  ]
}
```

Now using the ETL process, we loaded the flat file data into staging tables. Intermediate tables will look like below:

Name	Mem_Id	Enroll_Dt	Flight_Dt	TIER	Agent_Name	State	County	DOB	FLAG
Elena	223457	20101012	20121013	GLD	Sam	CA	USA	3051985	A
Ravi	223458	20101012	20121013	SLV		TN	IND	3051985	A
Mateo	223459	20101012	20121013	GLD		NCR	PHIL	3051985	A
Nora	22345	20101012	20121013	PLT		ONT	CAN	3051985	A
Jacob	2256	20101012	20121013	SLV		VIC	AU	3051985	A

- All members related to India will go to Table_India, and so on for each country.
- Create the process considering we are getting billions of records every day, across both the flat file and the JSON feed.

Technical Assessment: Deliverables

1. Create table queries – DDL for the raw/landing table, the staging table, and the country-specific target tables (e.g., in Snowflake).
2. Load the staging table with additional derived columns: Age (computed from DOB) and a Stale_Member flag where days since Flight_Date > 90.
3. Write the transformation logic (SQL and/or Python) to split members into their per-country target tables, applying the “latest record wins” rule when a member has moved countries.
4. Parse the semi-structured JSON redemption feed into a flattened, queryable table, and describe how you would join it back to the member profile data.
5. Create the necessary data validations — mandatory field checks, key-column uniqueness, and any checks you would add to catch the kind of data issues visible in the sample data above.
6. If we move forward with an interview, we would like to see a live demonstration.